

ΤΕΧΝΟΛΟΓΙΕΣ ΚΑΙ ΥΠΗΡΕΣΙΕΣ ΔΙΑΔΙΚΤΥΟΥ

ΕΡΓΑΣΙΑ ΜΕ ΧΡΗΣΗ ΤΩΝ MININET NETWORK EMULATOR ΚΑΙ WIRESHARK

Υπεύθυνη Εργασίας: Άννα Κεφάλα

(Ακαδημαϊκό έτος 2025-2026)

Ανάλυση Κυκλοφορίας σε Δίκτυο Multi-Hop με Κλασική Δρομολόγηση

Διαδικαστικά

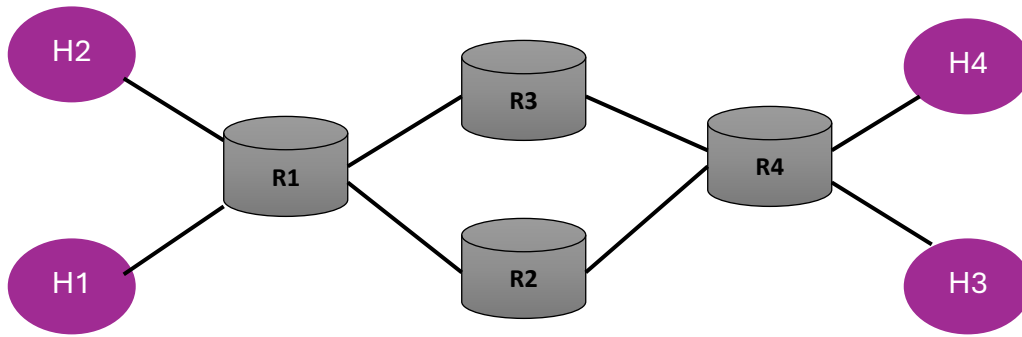
Η εργασία είναι **ατομική**. Θα πρέπει να υποβάλετε τις απαντήσεις σας μέχρι τις **18 Ιανουαρίου 2026**, μέσω του εργαλείου «Υποβολή Εργασιών» του e-class. Οι απαντήσεις σας θα πρέπει να περιέχονται σε ένα έγγραφο σε μορφή PDF, τεκμηριωμένες με περιγραφή της διαδικασίας που ακολουθήσατε, με χρήση κατάλληλων screenshots.

Στόχος

Χρήση του Mininet για τη σχεδίαση και υλοποίηση ενός δικτύου, το οποίο απαιτεί δρομολόγηση L3 ανάμεσα σε διαφορετικά υποδίκτυα. Θα χρησιμοποιήσετε αρχικά στατική δρομολόγηση και μετά δυναμική δρομολόγηση με το πρωτόκολλο RIP. Επιπλέον, αξιοποίηση του Wireshark στα Interfaces των δρομολογητών ώστε να καταγράψετε την αντίστοιχη κίνηση.

Βήματα Υλοποίησης – Ζητούμενα

1. **Σχεδιασμός Τοπολογίας:** Σχεδιάστε μια τοπολογία τύπου "**Διαμαντιού**" (H1-R1-R2-R4-H3 και H2-R1-R3-R4-H4) με hosts (Hi) και routers (Ri). Αναθέστε διευθύνσεις IP σε διαφορετικά υποδίκτυα για κάθε σύνδεση router-to-router και host-to-router (π.χ., 10.0.1.0/24, 10.0.2.0/24, κ.ο.κ.), σύμφωνα με το σχήμα:



2. **Υλοποίηση στο Mininet:** Δίνεται το Python script `routerline.py` το οποίο δημιουργεί τοπολογία αλυσίδας, τύπου H1-R1-R2-...-RN-H2. Δημιουργήστε με βάση αυτό το Python script τη ζητούμενη τοπολογία. Για την υλοποίηση των δρομολογητών χρησιμοποιούνται hosts ως L3 routers (χρήση Linux kernel routing).

3. Ρύθμιση Δρομολόγησης:

Σενάριο 1

Ρυθμίστε **στατικές διαδρομές** σε κάθε router ώστε το H1 να μπορεί να επικοινωνήσει με το H3 (διαδρομή: H1-R1-R2-R4-H3) και ο H2 να μπορεί να επικοινωνήσει με το H4 (διαδρομή: H2-R1-R3-R4-H4). ([τροποποιήστε αναλόγως το python script `routerline.py`](#) → [myrouterstopo.py](#))

Σενάριο 2

Χρησιμοποιήστε την απλή υλοποίηση του πρωτοκόλλου RIP (python script `rip.py`) ώστε να τρέξετε το πρωτόκολλο στην τοπολογία σας. Προκειμένου να τρέξει το πρωτόκολλο θα πρέπει να ανοίξετε (μέσα στο mininet) xterm σε κάθε router και να εκτελέσετε το script (`python3 rip.py &`).

4. Έλεγχος και Καταγραφή (Wireshark):

Σενάριο 1

- Εκκινήστε το Mininet εκτελώντας το python script σας:
(*Σημείωση: μπορείτε να έχετε τα scripts στο φάκελο με τα shared docs με το hosting OS και να ανοίξετε τερματικό στο φάκελο αυτό ώστε να τα τρέξετε*)
> `python3 myrouterstopo.py`
- Στο περιβάλλον του **Mininet CLI** τρέξτε τις ακόλουθες εντολές (λαμβάνοντας τα αντίστοιχα screenshots εκτέλεσής τους):
 - `mininet> dump`
 - `mininet> net`

- Εκκινήστε το **Wireshark** στον R1 (σε όλα τα interfaces του R1 - any). Για να εκκινήσετε το Wireshark, ανοίγετε xterm στον αντίστοιχο host/router (μέσα από το mininet CLI) και τρέχετε το Wireshark στον αντίστοιχο host/router.
- Επιβεβαιώστε τη συνδεσιμότητα δίνοντας την εντολή
mininet> [pingall](#)
- Ελέγξτε την καταγραφή στο Wireshark:
 - Αναλύστε το ARP traffic.
 - Αναλύστε το ICMP traffic (για ποιους κόμβους περνάει το ping, για ποιους όχι και γιατί;)

Σενάριο 2

- Εκκινήστε το Mininet εκτελώντας το python script σας
> [python3 myrouterstopo.py](#)
 - Εκκινήστε το **Wireshark** στον R1 (σε όλα τα interfaces του R1 - any).
 - Σε κάθε router της τοπολογίας σας (μέσα από το **Mininet CLI**) ανοίξτε xterm και εκτελέσετε το script
> [python3 rip.py &](#)
 - Ελέγξτε την καταγραφή στο Wireshark:
 - Αναλύστε το RIPv2 traffic (τί είδους μηνύματα στέλνονται/λαμβάνονται, από ποιους κόμβους λαμβάνονται μηνύματα, κλπ.).
 - Σε ποιο port επικοινωνεί το RIP; Ποιο πρωτόκολλο επιπέδου μεταφοράς χρησιμοποιεί;
5. **Τεκμηρίωση:** Καταγράψτε τη διαδικασία, την τοπολογία, τις ρυθμίσεις δρομολόγησης και τα screenshots από Mininet και Wireshark με κατάλληλα σχόλια. Επισυνάψτε τα python scripts με αντίστοιχα σχόλια στα κομμάτια του κώδικα που συμπληρώσατε/αλλάξατε.

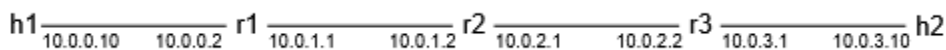
Ακολουθεί συνοδευτική τεκμηρίωση των *python scripts* που δίνονται για την υλοποίηση του *project*.

IP Routers σε μια γραμμή (αλυσίδα) – [routerline.py](#)

Υλοποίηση τοπολογίας στο mininet που περιλαμβάνει δρομολογητές για κλασσική δρομολόγηση, αντί για SDN switches. Ένας δρομολογητής είναι απλά ένας multi-interface Mininet host που έχει ενεργοποιημένο IP forwarding στο Linux kernel.

Η πρώτη τοπολογία δρομολογητή έχει μόνο δύο hosts, έναν σε κάθε άκρο, και N-routers ενδιάμεσα. Το παρακάτω διάγραμμα θεωρεί N=3. Όλα τα υποδίκτυα είναι /24. Το πρόγραμμα μπορεί να εκτελεστεί δίνοντας ως παράμετρο διαφορετικό N:

➤ **python3 routerline.py -N 3**



Οι καταχωρήσεις δρομολόγησης δημιουργούνται για να δρομολογήσουν την κυκλοφορία από το h1 στο h2, αλλά όχι αμφίδρομα. Δηλαδή, κάθε δρομολογητής έχει διαδρομή προς το 10.0.3.0/24, αλλά μόνο ο r1 ξέρει πώς να φτάσει στο 10.0.0./24 (σε ποια διεπαφή του r1 συνδέεται άμεσα).

Για να φτάσουμε από το h1 στο h2 πρέπει να οριστούν σε r1 και r2 πώς φτάνουμε στον προορισμό 10.0.3.0/24 με τις ακόλουθες εντολές (οι οποίες εκτελούνται αυτόματα αν οριστεί `ENABLE_LEFT_TO_RIGHT_ROUTING = True` στο python script):

```
r1: ip route add to 10.0.3.0/24 via 10.0.1.2
r2: ip route add to 10.0.3.0/24 via 10.0.2.2
```

Για αμφίδρομη σύνδεση θα πρέπει να οριστούν οι ακόλουθες διαδρομές προς το 10.0.0.0/24:

```
r2: ip route add to 10.0.0.0/24 via 10.0.1.1
r3: ip route add to 10.0.0.0/24 via 10.0.2.1
```

Κατά την υλοποίηση της τοπολογίας του δικτύου, για τους hosts (κόμβους με μία διεπαφή) μπορούμε να ορίσουμε όλα τα χαρακτηριστικά τους ταυτόχρονα:

```
h1 = self.addHost('h1', ip='10.0.0.10/24', defaultRoute='via 10.0.0.2')
h2 = self.addHost('h2', ip='10.0.3.10/24', defaultRoute='via 10.0.3.1')
```

Οι δρομολογητές δημιουργούνται επίσης με το `addHost()`, αλλά με ξεχωριστές εντολές:

```
r1 = self.addHost( 'r1' )
r2 = self.addHost( 'r2' )
...
```

```
self.addLink( h1, r1, intfName1 = 'h1-eth0', intfName2 = 'r1-eth0')
self.addLink( r1, r2, inftname1 = 'r1-eth1', inftname2 = 'r2-eth0')
Στη συνέχεια οι δρομολογητές λαμβάνουν τις διευθύνσεις IPv4 τους:
```

```
r1 = net['r1']
r1.cmd('ifconfig r1-eth0 10.0.0.2/24')
r1.cmd('ifconfig r1-eth1 10.0.1.1/24')
r1.cmd('sysctl net.ipv4.ip_forward=1')
rp_disable(r1)
```

Η εντολή **sysctl** επιτρέπει την προώθηση στο r1. Η κλήση **rp_disable (r1)** απενεργοποιεί την προεπιλεγμένη συμπεριφορά του Linux να μην προωθεί πακέτα αν ο δρομολογητής δεν έχει μια διαδρομή πίσω προς την πηγή του πακέτου.

IP Routers με απλό Distance-Vector αλγόριθμο δρομολόγησης (RIP) – [rip.py](#)

Δρομολόγηση με βάση αλγόριθμο ανακάλυψης της διαδρομής από το h1 στο h2 (και αντίστροφα). Υλοποίηση βασισμένη σε απλή εφαρμογή του RIP.

Εν συντομία, η ιδέα είναι να προσθέσουμε ένα πεδίο «κόστος» στον πίνακα προώθησης, έτσι ώστε οι εγγραφές να έχουν τη μορφή (*destination, next_hop, cost*). Στη συνέχεια, οι δρομολογητές στέλνουν τα διανύσματα (*destination, cost*) στους γείτονές τους (*update messages* σύμφωνα με το RIP). Οι δρομολογητές που λαμβάνουν αυτά τα μηνύματα, τα επεξεργάζονται για να υπολογίσουν τη διαδρομή χαμηλότερου κόστους για κάθε προορισμό. Η μορφή των μηνυμάτων ενημέρωσης είναι η εξής:

Addr Family	route_tag
IP Address	
Netmask	
Next_hop Address	
metric	

Το πρόγραμμα περιμένει να λάβει μηνύματα ενημέρωσης από άλλους δρομολογητές. Κάθε UPDATE_INTERVAL (δευτερόλεπτα) το πρόγραμμα στέλνει τα δικά του μηνύματα ενημέρωσης. Όλη η επικοινωνία πραγματοποιείται μέσω των πακέτων UDP που αποστέλλονται χρησιμοποιώντας IP multicast, στην επίσημη διεύθυνση RIP multicast 224.0.0.9. Το port 520 χρησιμοποιείται τόσο για την αποστολή όσο και για τη λήψη.

Αντί να δημιουργούνται ξεχωριστά threads για λήψη και αποστολή, ρυθμίζεται ένα σύντομο (1 δευτερόλεπτο) *recv()* timeout και στη συνέχεια μετά από κάθε timeout, ελέγχουμε αν είναι ώρα για να στείλουμε την επόμενη ενημέρωση.

Το πρόγραμμα διατηρεί ένα αντίγραφο (shadow copy) RTable του πραγματικού πίνακα προώθησης του συστήματος, με μια στήλη πρόσθετου κόστους. Ο πραγματικός πίνακας ενημερώνεται κάθε φορά που αλλάζει μια διαδρομή στον shadow copy RTable. Στο πρόγραμμα, το RTable είναι ένα dictionary αντιστοίχισης TableKey values (αποτελούμενη από τη διεύθυνση IP και τη μάσκα) σε TableValue objects που περιέχουν interface name, cost, και next_hop.